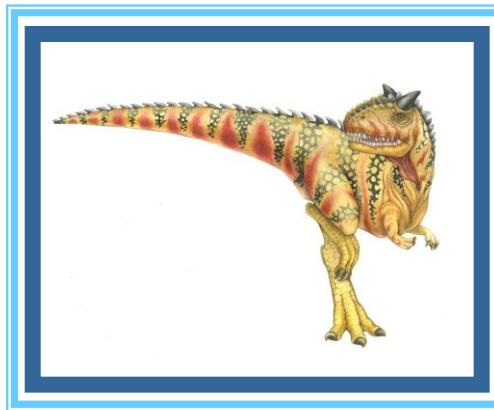


Chapter 3: Processes





Chapter 3: Processes

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication





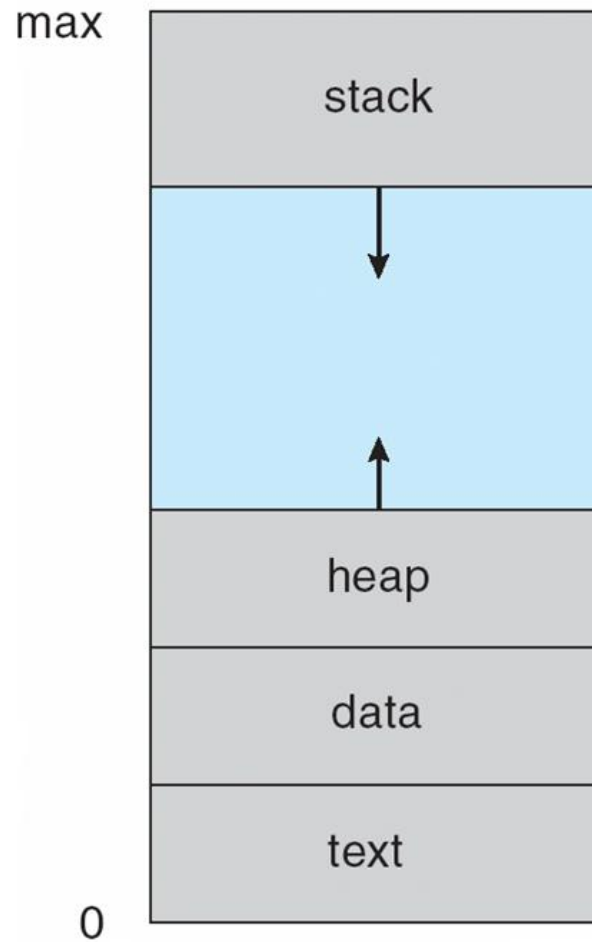
Process Concept

- ❑ An operating system executes a variety of programs:
 - ❑ Batch system – **jobs**
 - ❑ Time-shared systems – **user programs** or **tasks**
- ❑ terms **job** and **process** used interchangeably
- ❑ **Process** – a program in execution; process execution must progress in sequential fashion
- ❑ Multiple parts
 - ❑ The program code, also called **text section**
 - ❑ Current activity including **program counter**, processor registers
 - ❑ **Stack** containing temporary data
 - ▶ Function parameters, return addresses, local variables
 - ❑ **Data section** containing global variables
 - ❑ **Heap** containing memory dynamically allocated during run time





Process in Memory





Process Concept (Cont.)

- ❑ Program is **passive** entity stored on disk (**executable file**), process is **active**
 - ❑ Program becomes process when executable file loaded into memory
- ❑ Execution of program started via GUI mouse clicks, command line entry of its name, etc
- ❑ One program can be several processes
 - ❑ Consider multiple users executing the same program
- ❑ Although two processes may be associated with the same program, they are nevertheless considered two separate execution sequences.
- ❑ For instance, several users may be running different copies of the mail program, or the same user may invoke many copies of the Web browser program.
- ❑ Each of these is a separate process, and although the text sections are equivalent, the data, heap, and stack sections vary.
- ❑ It is also common to have a process that spawns many processes as it runs





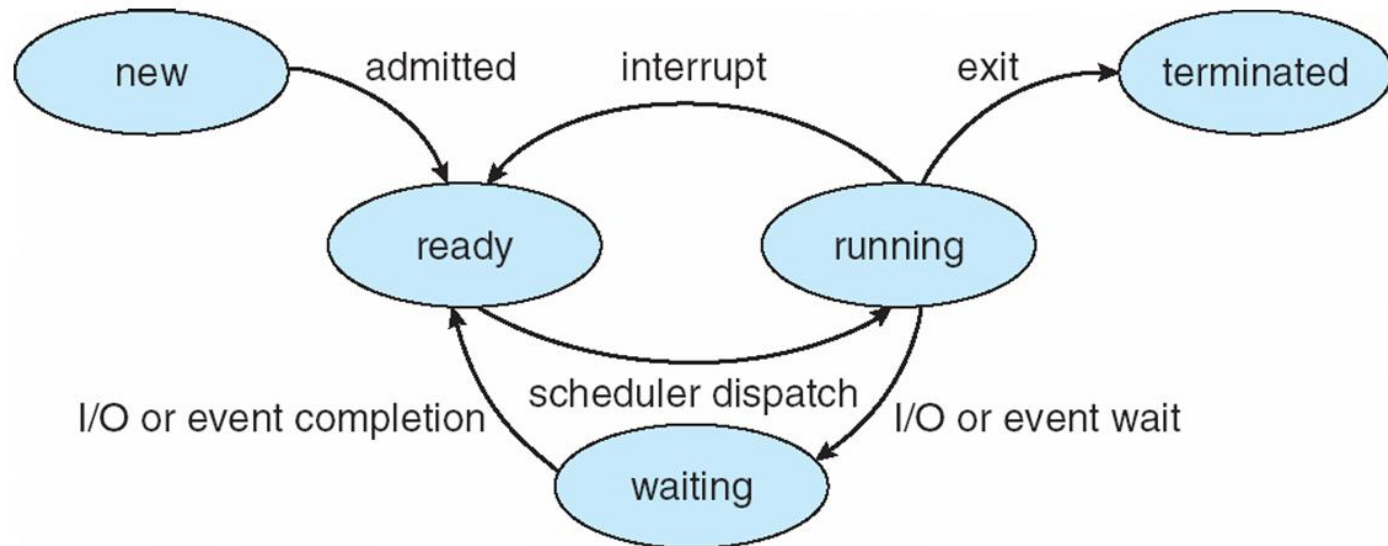
Process State

- As a process executes, it changes **state**
 - **new**: The process is being created
 - **running**: Instructions are being executed
 - **waiting**: The process is waiting for some event to occur
 - **ready**: The process is waiting to be assigned to a processor
 - **terminated**: The process has finished execution
- **only one process can be *running* on any processor at any instant.**
- Many processes may be *ready* and *waiting*, however





Diagram of Process State



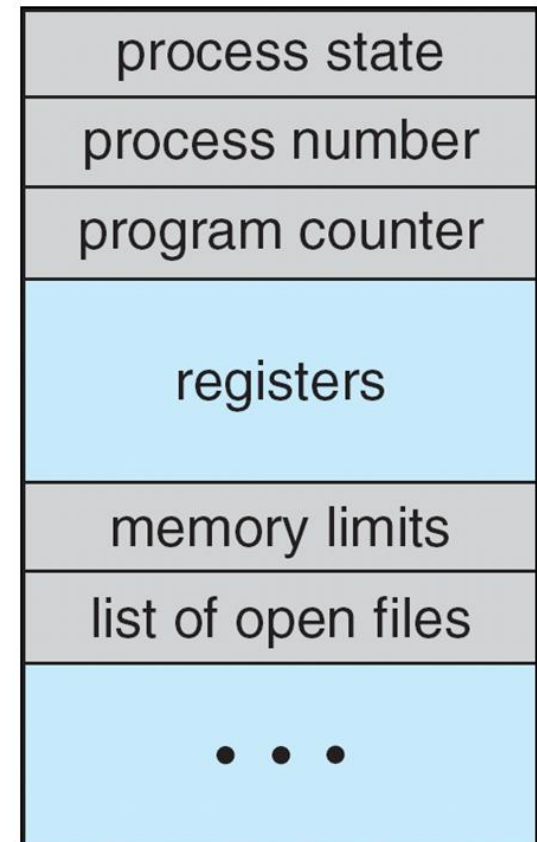


Process Control Block (PCB)

Information associated with each process

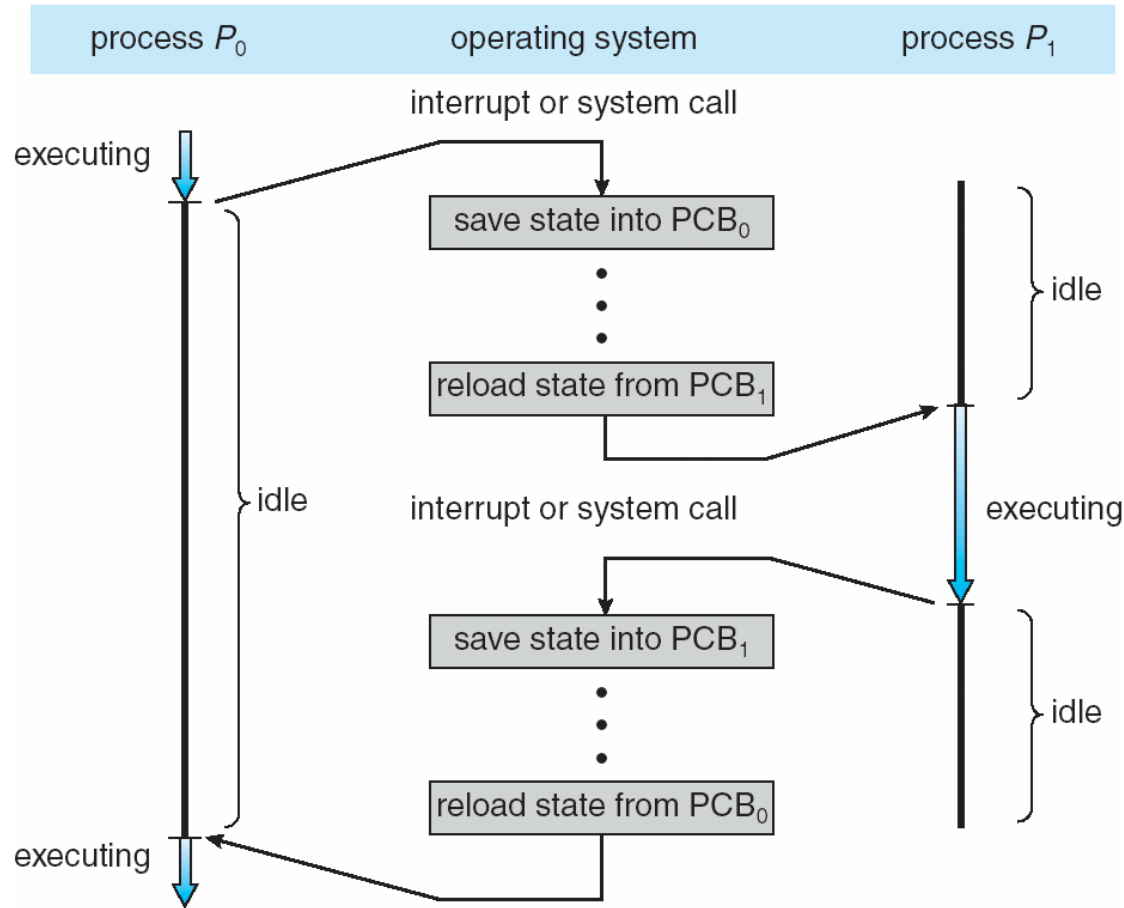
(also called **task control block**)

- ❑ Process state – running, waiting, etc
- ❑ Program counter – location of instruction to next execute
- ❑ CPU registers – contents of all process-centric registers
- ❑ CPU scheduling information- priorities, scheduling queue pointers
- ❑ Memory-management information – memory allocated to the process, information such as the value of the base and limit registers, the page tables, or the segment tables, depending on the memory system used by the operating system
- ❑ Accounting information – CPU used, clock time elapsed since start, time limits, acc. No., process no. etc
- ❑ I/O status information – I/O devices allocated to process, list of open files





CPU Switch From Process to Process





Threads

- ❑ In the model discussed so far, process has a single thread of execution
- ❑ This single thread of control allows the process to perform only one task at a time
- ❑ Many modern operating systems have extended the process concept to allow a process to have multiple threads of execution and thus to perform more than one task at a time.
- ❑ On a system that supports threads, the PCB is expanded to include information for each thread.
- ❑ Consider having multiple program counters per process
 - ❑ Multiple locations can execute at once
 - ▶ Multiple threads of control -> **threads**
- ❑ Must then have storage for thread details, multiple program counters in PCB





Process Scheduling

- ❑ The objective of **multiprogramming** is to have some process running at all times, to maximize CPU utilization
- ❑ The objective of **time sharing** is to switch the CPU among processes so frequently that users can interact with each program while it is running
- ❑ **Process scheduler** selects among available processes for next execution on CPU
- ❑ For a single-processor system, there will never be more than one running process.
- ❑ If there are more processes, the rest will have to wait until the CPU is free and can be rescheduled





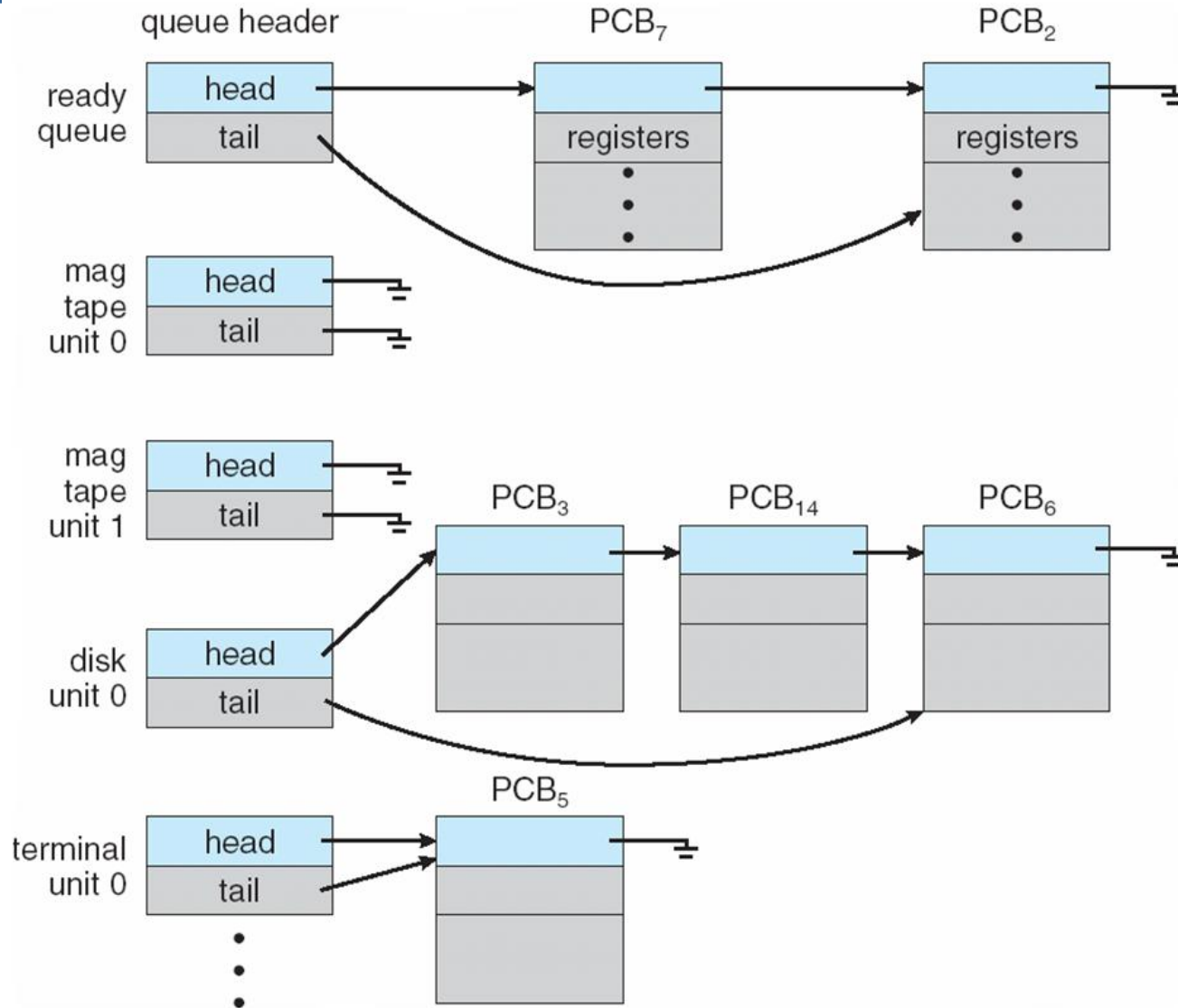
Scheduling queues

- Maintains **scheduling queues** of processes
 - **Job queue** – set of all processes in the system
 - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
 - ▶ stored as a linked list.
 - ▶ A ready-queue header contains pointers to the first and final PCBs in the list.
 - ▶ Each PCB includes a pointer field that points to the next PCB in the ready queue
 - **Device queues** – set of processes waiting for an I/O device
 - ▶ Each device has its own device queue
 - Processes migrate among the various queues





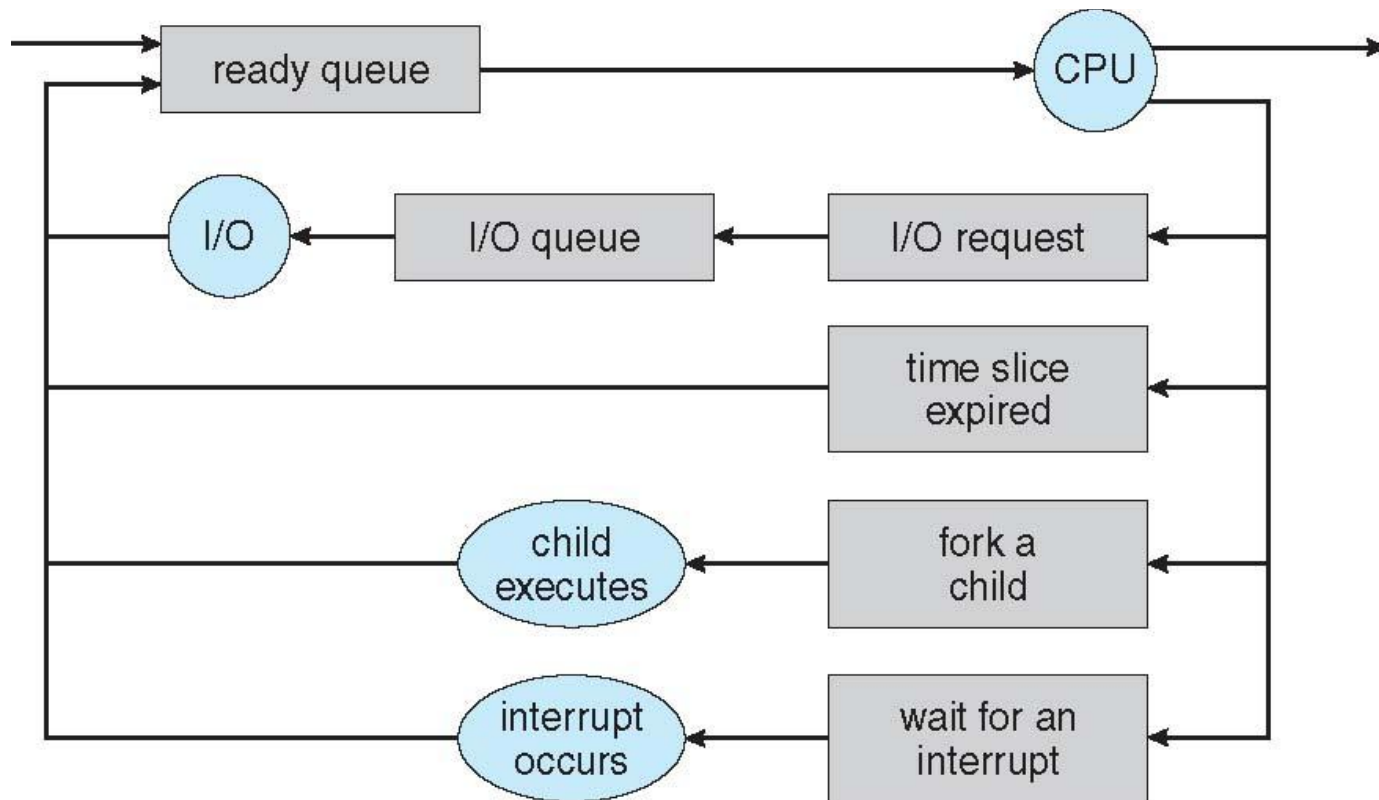
The ready queue and various I/O device queues.





Queueing-diagram representation of process scheduling

- **Queueing diagram** represents queues, resources, flows

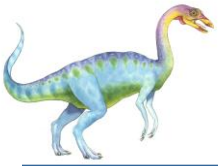




Schedulers

- ❑ **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates CPU
 - ❑ Sometimes the only scheduler in a system
 - ❑ Short-term scheduler is invoked frequently (milliseconds) $\Rightarrow$ (must be fast)
- ❑ **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
 - ❑ In batch processing, more processes are submitted than can be executed immediately.
 - ❑ These processes are spooled to a mass-storage device
 - ❑ Long-term scheduler is invoked infrequently (seconds, minutes) $\Rightarrow$ (may be slow)
 - ❑ The long-term scheduler controls the **degree of multiprogramming (the number of processes in memory)**
 - ❑ If the degree of multiprogramming is stable, then the average rate of process creation must be equal to the average departure rate of processes leaving the system.
 - ❑ Thus, the long-term scheduler may need to be invoked only when a process leaves the system.
 - ❑ Because of the longer interval between executions, the long-term scheduler can afford to take more time to decide which process should be selected for execution.





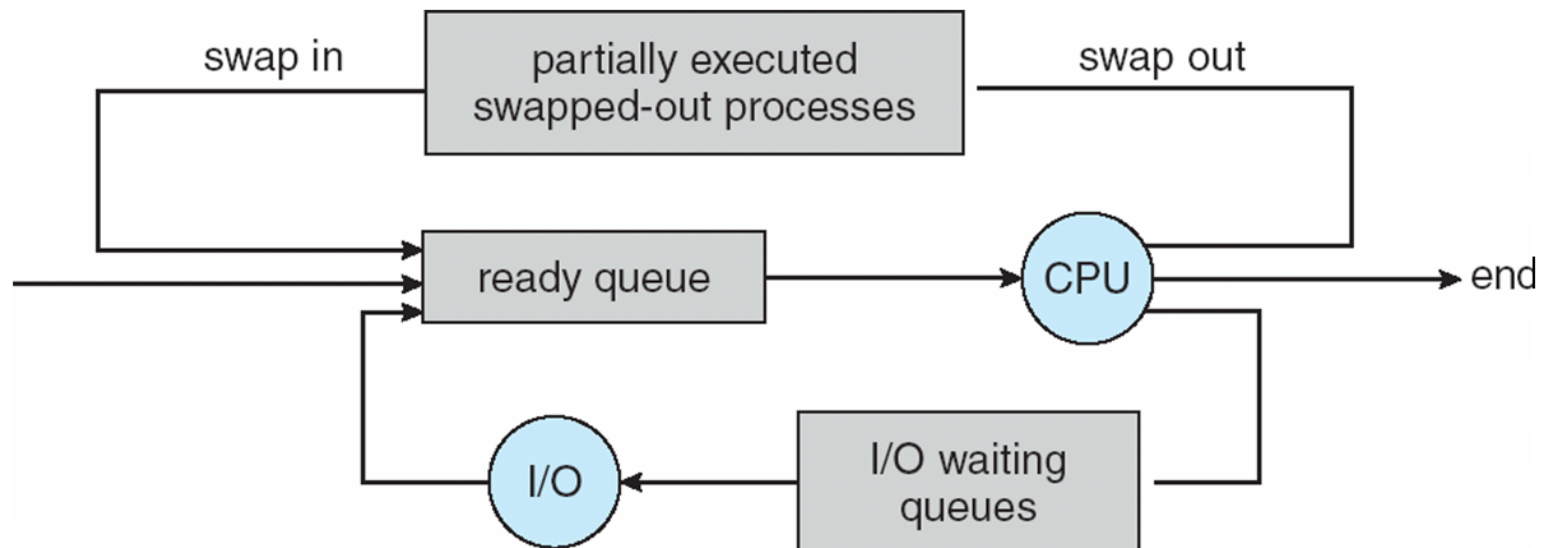
- Processes can be described as either:
 - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
 - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
- Long-term scheduler strives for good ***process mix***





Addition of Medium Term Scheduler

- **Medium-term scheduler** can be added if degree of multiple programming needs to decrease
 - Remove process from memory, store on disk, bring back in from disk to continue execution: **swapping**





Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
 - The more complex the OS and the PCB → the longer the context switch
- Time dependent on hardware support
 - Some hardware provides multiple sets of registers per CPU → multiple contexts loaded at once





Operations on Processes

- System must provide mechanisms for:
 - process creation,
 - process termination,





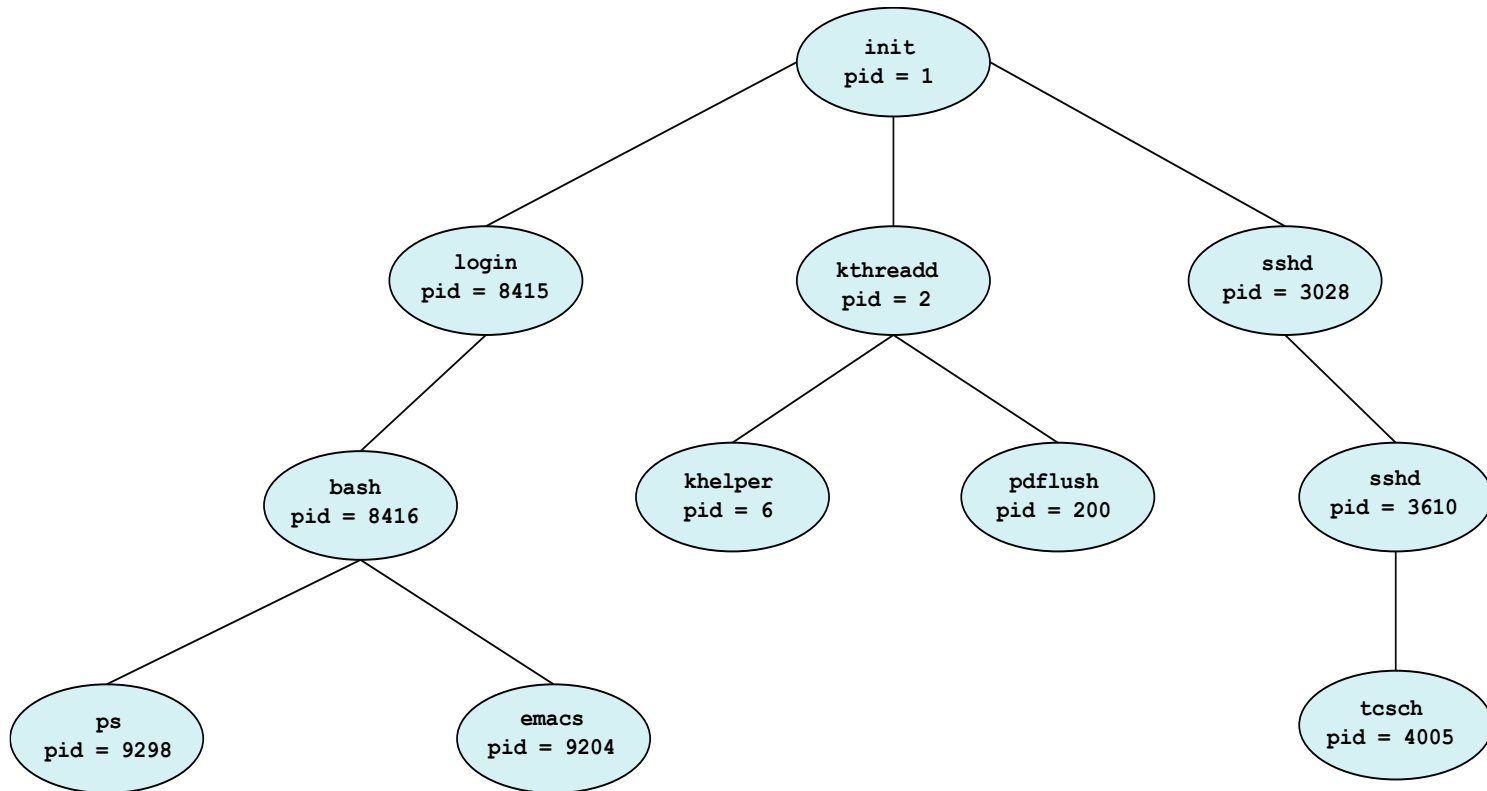
Process Creation

- ❑ process may create several new processes, via a create-process system call, during the course of execution
- ❑ The creating process is called a **parent** process, and the new processes are called the **children** of that process
- ❑ Each of these new processes may in turn create other processes, forming a **tree** of processes
- ❑ Generally, process is identified and managed via a **process identifier (pid)** – large integer
- ❑ Resource sharing options
 - ❑ Parent and children share all resources
 - ❑ Children share subset of parent's resources
 - ❑ Parent and child share no resources
- ❑ Execution options
 - ❑ Parent continues to execute concurrently with its children
 - ❑ Parent waits until some or all of its children have terminated





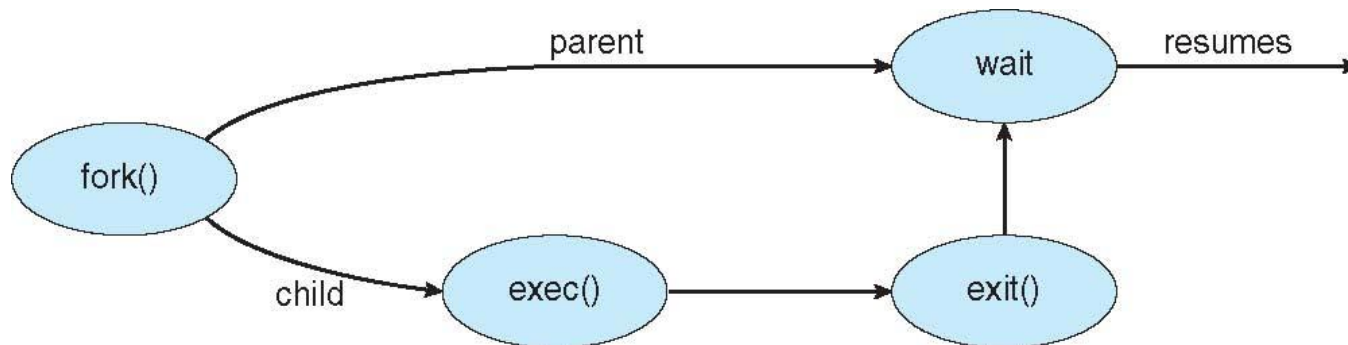
A Tree of Processes in Linux





Process Creation (Cont.)

- Address space
 - Child duplicate of parent (it has the same program and data as the parent)
 - Child process has a new program loaded into it
- UNIX examples
 - **fork()** system call creates new process
 - **exec()** system call used after a **fork()** to replace the process' memory space with a new program





C Program Forking Separate Process

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

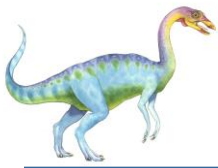
int main()
{
    pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }

    return 0;
}
```





Process Termination

- terminates when it finishes executing its final statement and then asks the operating system to delete it using the **exit()** system call.
 - Returns status data from child to parent (via **wait()**)
 - Process' resources (physical and virtual memory, open files, and I/O buffers) are deallocated by operating system
- Parent may terminate the execution of children processes using the **abort()** system call. Some reasons for doing so:
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - The parent is exiting and the operating systems does not allow a child to continue if its parent terminates





Process Termination

- ❑ Some operating systems do not allow child to exist if its parent has terminated. If a process terminates, then all its children must also be terminated.
 - ❑ **cascading termination.** All children, grandchildren, etc. are terminated.
 - ❑ The termination is initiated by the operating system.
- ❑ The parent process may wait for termination of a child process by using the `wait()` system call. The call returns status information and the pid of the terminated process

```
pid = wait(&status);
```
- ❑ If the parent terminates, however, all its children have assigned as their new parent the init process.
- ❑ Thus, the children still have a parent to collect their status and execution statistics





Interprocess Communication

- Processes executing concurrently within a system may be ***independent*** or ***cooperating***
- A process is ***independent*** if it cannot affect or be affected by the other processes executing in the system.
 - Any process that does not share data with any other process is independent
- A process is ***cooperating*** if it can affect or be affected by the other processes executing in the system.
 - Any process that shares data with other processes is a cooperating process





Reasons for cooperating processes

- Information sharing -several users may be interested in the same piece of information (for instance, a shared file)
 - ▶ provide an environment to allow concurrent access to such information
- Computation speedup -If we want a particular task to run faster, we must break it into subtasks, each of which will be executing in parallel with the others
- Modularity -to construct the system in a modular fashion, dividing the system functions into separate processes or threads,
- Convenience - an individual user may work on many tasks at the same time
 - ▶ user may be editing, printing, and compiling in parallel





Interprocess communication (IPC)

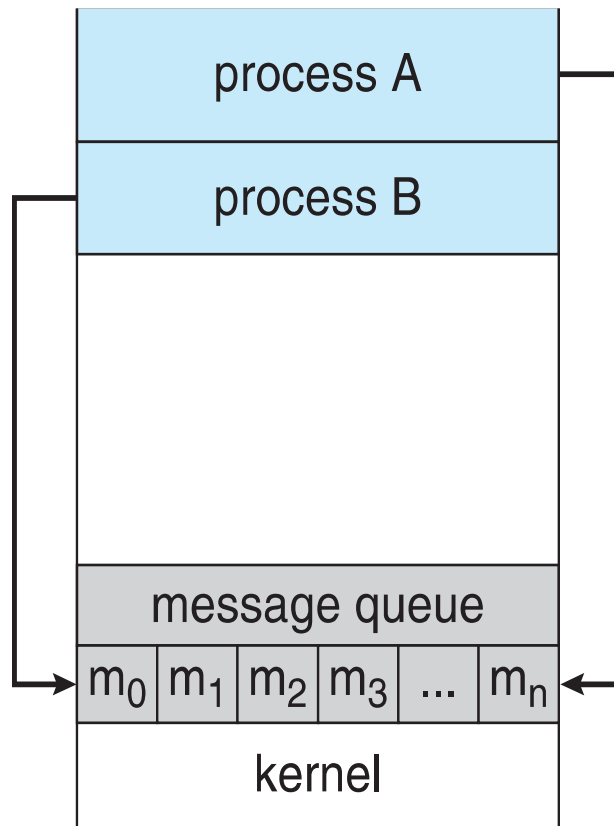
- Cooperating processes need **interprocess communication (IPC)**
- Two models of IPC
 - **Shared memory** - region of memory that is shared by cooperating processes is established.
 - ▶ Processes can then exchange information by reading and writing data to the shared region
 - **Message passing** - , communication takes place by means of messages exchanged between the cooperating processes.



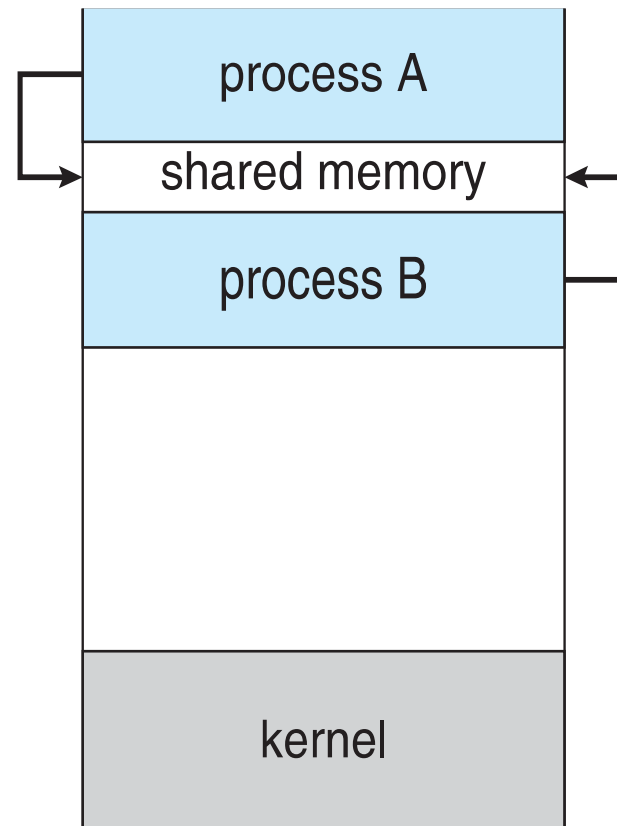


Communications Models

(a) Message passing. (b) shared memory.

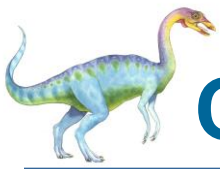


(a)



(b)





Communications Models Contd...

- ❑ **Message passing** is useful for exchanging smaller amounts of data, because no conflicts need be avoided.
- ❑ **Message passing** is also easier to implement than shared memory for intercomputer communication.
- ❑ **Shared memory** allows maximum speed and convenience of communication.
- ❑ **Shared memory** is faster than message passing, as message passing systems are typically implemented using system calls and thus require the more time-consuming task of kernel intervention.
- ❑ In contrast, in **shared memory systems**, system calls are required only to establish shared-memory regions.
- ❑ Once shared memory is established, all accesses are treated as routine memory accesses, and no assistance from the kernel is required.





Producer-Consumer Problem

- Paradigm for cooperating processes, *producer* process produces information that is consumed by a *consumer* process
 - **unbounded-buffer** places no practical limit on the size of the buffer
 - **bounded-buffer** assumes that there is a fixed buffer size





Bounded-Buffer – Shared-Memory Solution

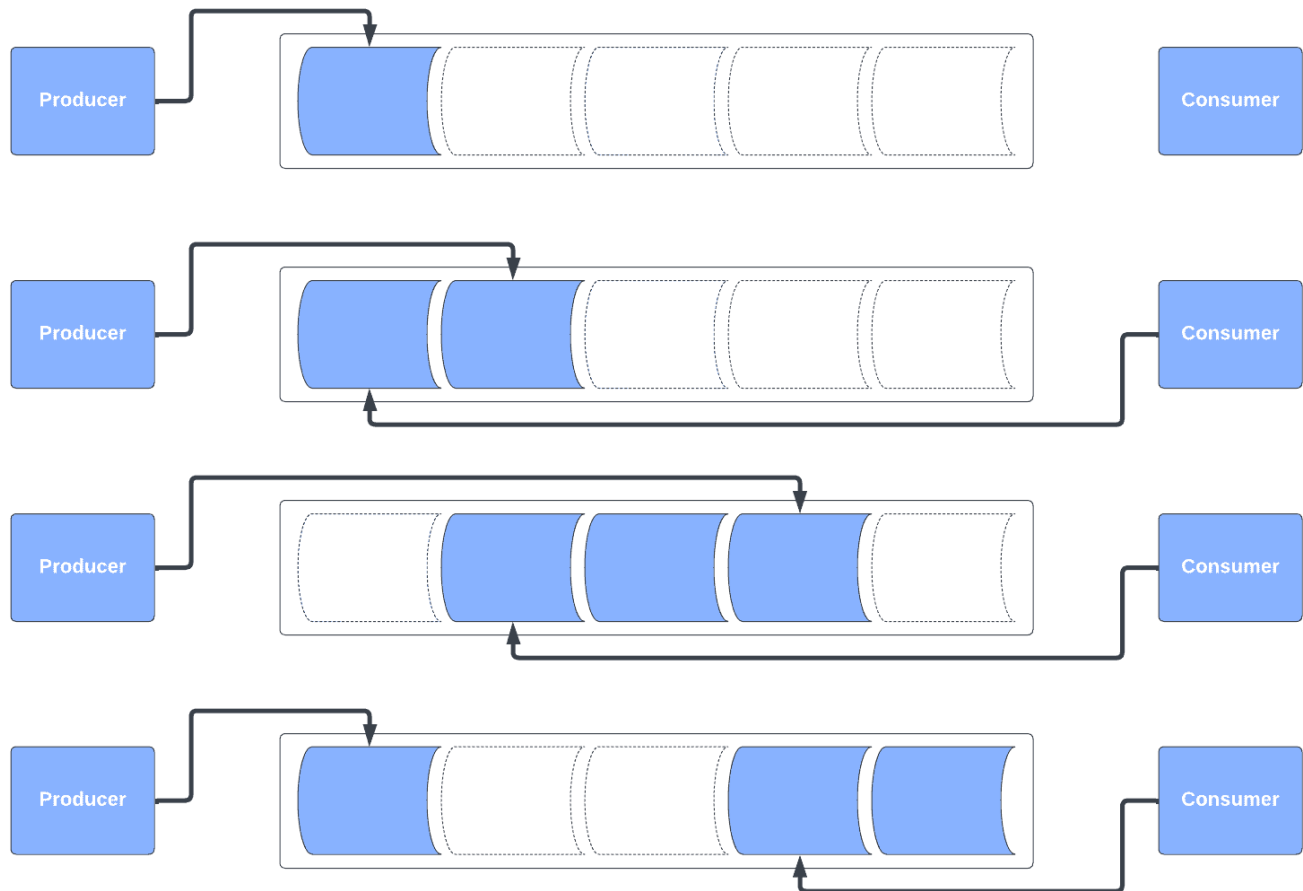
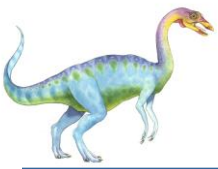
□ Shared data

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

□ Solution is correct, but can only use BUFFER_SIZE-1 elements





[The Bounded-Buffer Problem | Baeldung on Computer Science](#)





Step 1: Initial State

- Buffer: All slots are empty.
- Pointers: $in = 0$, $out = 0$
- Condition: $in == out \rightarrow$ Buffer is empty.
- Action: Producer can insert; consumer must wait.

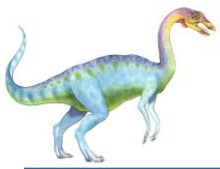




Step 2: Producer Inserts Item

- Action: Producer places an item at index in .
- Update: $in = (in + 1) \% size$
- Buffer: One slot filled.
- Condition: Buffer is not full, consumer can consume.

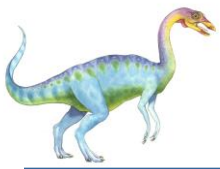




Step 3: Consumer Removes Item

- Action: Consumer takes item from index out.
- Update: $out = (out + 1) \% size$
- Buffer: One slot emptied.
- Condition: Buffer is not empty, producer can continue.





Step 4: Wrap-Around Behavior

- When in or out reaches the end of the array, it wraps to index 0.
- Achieved using modulo operation: $(\text{pointer} + 1) \% \text{size}$

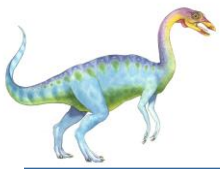




Step 5: Buffer Full Condition

- Condition: $(in + 1) \% size == out$
- Action: Producer must wait.
- Buffer: All slots filled except one reserved to distinguish full from empty.





Step 6: Buffer Empty Condition

- Condition: $in == out$
- Action: Consumer must wait.
- Buffer: All slots empty.





Bounded-Buffer – Producer

```
item next_produced;
while (true) {
    /* produce an item in next produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```





Bounded Buffer – Consumer

```
item next_consumed;
while (true) {
    while (in == out)
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next consumed */
}
```





Interprocess Communication – Shared Memory

- ❑ An area of memory shared among the processes that wish to communicate
- ❑ The communication is under the control of the users processes not the operating system.
- ❑ The scheme requires that these processes share a region of memory and that the code for accessing and manipulating the shared memory be written explicitly by the application programmer
- ❑ Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.





Interprocess Communication – Message Passing

- Mechanism for processes to communicate and to synchronize their actions
- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
 - `send(message)`
 - `receive(message)`
- The *message* size is either fixed or variable
- If only fixed-sized messages can be sent, the system-level implementation is straightforward.
 - makes the task of programming more difficult
- variable-sized messages require a more complex system-level implementation,
 - programming task becomes simpler





Message Passing (Cont.)

- If processes P and Q wish to communicate, they need to:
 - Establish a **communication link** between them
 - Exchange messages via send/receive
- Implementation issues:
 - How are links established?
 - Can a link be associated with more than two processes?
 - How many links can there be between every pair of communicating processes?
 - What is the capacity of a link?
 - Is the size of a message that the link can accommodate fixed or variable?
 - Is a link unidirectional or bi-directional?

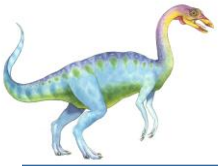




Message Passing (Cont.)

- Implementation of communication link
 - Physical:
 - ▶ Shared memory
 - ▶ Hardware bus
 - ▶ Network
 - Logical:
 - ▶ Direct or indirect
 - ▶ Synchronous or asynchronous
 - ▶ Automatic or explicit buffering





Naming

- Processes that want to communicate must have a way to refer to each other. They can use either **direct** or **indirect** communication.

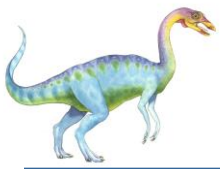




Direct Communication

- In **direct** communication, Processes must name each other explicitly:
 - **send** (P , $message$) – send a message to process P
 - **receive**(Q , $message$) – receive a message from process Q
- Properties of communication link
 - Links are established automatically between every pair of processes that want to communicate.
 - The processes need to know only each other's identity to communicate
 - A link is associated with exactly one pair of communicating processes
 - Between each pair there exists exactly one link





Direct Communication Contd...

- exhibits **symmetry** in addressing
 - both the sender process and the receiver process must name the other to communicate
- variant of this scheme - **asymmetry** in addressing.
 - only the sender names the recipient; the recipient is not required to name the sender.
- `send()` and `receive()` primitives are defined as follows:
- **`send(P, message)`** —Send a message to process *P*.
- **`receive(id, message)`** —Receive a message from any process; the variable *id* is set to the name of the process with which communication has taken place
- The disadvantage in both of these schemes (symmetric and asymmetric)
 - limited modularity of the resulting process definitions.
 - Changing the identifier of a process may necessitate examining all other process definitions.
 - All references to the old identifier must be found, so that they can be modified to the new identifier





Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
 - Each mailbox has a unique id
 - Processes can communicate only if they share a mailbox
- Primitives are defined as:
 - send**(*A, message*) – send a message to mailbox A
 - receive**(*A, message*) – receive a message from mailbox A
- Properties of communication link
 - Link established only if processes share a common mailbox
 - A link may be associated with many processes
 - Between each pair of communicating processes, there may be a number of different links, with each link corresponding to one mailbox





Indirect Communication

- Mailbox sharing
 - P_1 , P_2 , and P_3 share mailbox A
 - P_1 , sends; P_2 and P_3 receive
 - Who gets the message?
- Solutions
 - Allow a link to be associated with at most two processes
 - Allow only one process at a time to execute a receive operation
 - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.





Indirect Communication

- mailbox may be **owned either by a process or by the operating system.**
- mailbox **owned by a process** - part of the address space of the process
 - owner (which can only receive messages through this mailbox)
 - user (which can only send messages to the mailbox).
 - Each mailbox has a unique owner, - no confusion about which process should receive a message sent to this mailbox.
 - When a process that owns a mailbox terminates, the mailbox disappears.
 - Any process that subsequently sends a message to this mailbox must be notified that the mailbox no longer exists.
- a mailbox that is owned by the **operating system** has an existence of its own.
 - It is independent and is not attached to any particular process.
 - The operating system then must provide a mechanism that allows a process to do the following
- Operations
 - create a new mailbox (port)
 - send and receive messages through mailbox
 - delete a mailbox





Synchronization

- ? Message passing may be either blocking or non-blocking
- ? **Blocking** is considered **synchronous**
 - ? **Blocking send** -- the sender is blocked until the message is received
 - ? **Blocking receive** -- the receiver is blocked until a message is available
- ? **Non-blocking** is considered **asynchronous**
 - ? **Non-blocking send** -- the sender sends the message and resumes operation
 - ? **Non-blocking receive** -- the receiver receives:
 - ? A valid message, or
 - ? Null message
- ? Different combinations possible
 - ? If both send and receive are blocking, we have a **rendezvous**





Synchronization (Cont.)

□ Producer-consumer becomes trivial

```
message next_produced;  
while (true) {  
    /* produce an item in next produced */  
    send(next_produced);  
}
```

```
message next_consumed;  
while (true) {  
    receive(next_consumed);  
  
    /* consume the item in next consumed */  
}
```

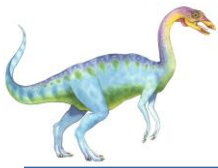




Buffering

- Whether communication is direct or indirect, messages exchanged by communicating processes reside in a temporary queue.
- Queue of messages attached to the link.
- implemented in one of three ways
 1. Zero capacity – maximum length of zero
 - ▶ no messages are queued on a link.
 - ▶ Sender must block until the recipient receives the message
 2. Bounded capacity – finite length of n messages
 - ▶ at most n messages can reside in it.
 - ▶ If the queue is not full when a new message is sent, the message is placed in the queue (either the message is copied or a pointer to the message is kept), and the sender can continue execution without waiting
 - ▶ , If the link is full Sender must block until space is available in the queue
 3. Unbounded capacity – infinite length
Sender never blocks

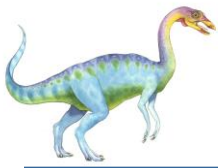




Acknowledgement

- This slides are borrowed from the ppt of Operating Systems Concepts 9Th Edition By Silberschatz, Galvin and Gagne with some modifications done





Reference

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, (9e), Wiley and Sons (Asia) Pvt. Ltd, 2016.



End of Chapter 3

